Weekly Progress Report

Refactoring Detection Project

Prepared for Monday research supervisor meeting

1. Overview

This week, the team focused on understanding the project, reviewing the system documentation, learning the multi-agent workflow, and starting the design process for the user interface. The main goal was to understand what the tool does, who it is for, and how the interface should communicate refactoring suggestions clearly to users.

2. Weekly Progress Summary

Area	Progress Made
Project onboarding	The team became familiar with the project scope and the main objective of the refactoring detection tool.
Documentation review	We reviewed the project PDF and studied the workflow described in the documentation.
System architecture	We identified the main agents and understood how they work together.
UI design research	We discussed the target audience and what the prototype should include.
Prototype creation	Each team member created a prototype idea for the tool interface.
Prototype selection	On Friday, the team compared the prototypes and selected the design direction to continue with.
GitHub collaboration	We started learning GitHub and how it can support team organization, version control, and task division.

3. What the Program Does

The program is designed to help Java developers when they copy and paste code. After a paste action, it checks whether the pasted code is duplicated or very similar to code that already exists in the same Java file.

If duplicated code is found, the tool suggests an Extract Method refactoring. This means it creates one helper method containing the repeated logic and replaces the repeated code blocks with calls

to that method. The goal is not to add new features, but to clean the code, reduce repetition, and make the project easier to maintain.

4. GitHub Setup and Team Collaboration

During the meeting, the team also got familiar with GitHub. GitHub will be used to keep the project files in one place, track changes, divide tasks, and review work before adding it to the final version.

GitHub Cheat Sheet

Command / Term	Meaning
git clone	Copy the project to your computer
git pull	Get the latest changes
git add .	Prepare changed files
git commit	Save a version of your changes
git push	Upload your changes
branch	Work separately without changing the main version
pull request	Ask the team to review and approve changes

5. Understanding the Multi-Agent System

The tool is built as a multi-agent system. Each agent has a specific role, and together they detect duplicates, suggest a refactoring, check if it is correct, and test whether the behavior stays the same.

Agent	Simple Explanation
Detection Agent	Checks the pasted code and searches for similar code blocks in the same Java file. It returns the clone locations or says that no clones were found.
Curator	Acts like a judge. It compares the answers from different AI agents and chooses the best and safest result.
Refactoring Agent	Takes the duplicated code and creates an Extract Method suggestion. It creates a helper method and replaces duplicate blocks with method calls.
Compilation Agent	Checks if the refactored Java code compiles. If there are errors, it sends feedback back to the Refactoring Agent.
Testing Agent	Runs tests to check if the refactored code still behaves the same. If tests fail, the suggestion is retried or rejected.

6. Target Audience and Design Requirements

The target users are Java developers and students who want to improve code quality. Since the tool changes code, the interface needs to be clear, fast, and trustworthy. Users should quickly

understand what was detected, where the duplicated code is, what will change, and whether the suggested refactoring is safe.

- Show the clone type and the file locations with line numbers.
- Explain the suggested Extract Method refactoring in simple words.
- Show a diff or preview of the change before the user applies it.
- Display validation results such as usefulness, compilation, and tests.
- Give the user control through actions like Apply, Edit, View Diff, and Reject.

7. Prototype Work and Final Design Direction

Each team member created a prototype for the interface. The team compared the designs and focused on which one best communicates the clone detection result, the proposed refactoring, and the validation status.

The selected direction is a terminal-style interface that gives developers the important information first: duplicate code detected, clone type, file locations, suggested helper method, validation status, and available actions. This design works because it supports the target audience: developers who want quick, clear, and trustworthy information without being distracted from their coding workflow.

8. Next Steps Before and After Monday

- Finalize and improve the chosen prototype.
- Divide the presentation slides among team members.
- Prepare a short explanation for each agent.
- Continue using GitHub to organize files, tasks, and changes.
- Work individually on a simple website to track team progress.
- Move from prototype discussion toward a clearer UI implementation plan.

9. Conclusion

Overall, the team made progress in understanding the project, the system architecture, and the design needs of the tool. The next focus is to clearly present this progress to the research supervisor, continue improving the selected prototype, and organize future development work through GitHub and the progress tracking website.